

N-BODY PARTICLE SIMULATION

ANGELA GROSS // CSCI-595 // HOMEWORK 2

OVERVIEW

Similarly to the square dgemm homework, the n-body particle simulation homework was also extremely difficult to finish. I had first approached this homework using quadtrees and the Barnes Hut algorithm, but decided to go with complete the assignment by “binning” the particles, instead. With Robert’s help, I was able to get the Serial, OpenMP, and MPI “binning” implementations completed with grades 100, 95.44, and 79.76, respectively, as shown below.

```
1 n = 500, simulation time = 0.062794 seconds
2 n = 1000, simulation time = 0.1282 seconds
3 n = 2000, simulation time = 0.27024 seconds
4 n = 4000, simulation time = 0.579699 seconds
5 n = 8000, simulation time = 1.22103 seconds
6
7 Serial code is O(N^2slope)
8 Slope estimates are : 1.029698 1.075845 1.101063 1.074723
9 Slope estimate for line fit is: 1.073956
10 Serial Grade = 100.00
11
12
```

```
55 TACC: Starting up job 5012959
56 TACC: Setting up parallel environment for MMAPICH2+mpispaaw.
57 TACC: Starting parallel tasks...
58 n = 80000, simulation time = 5.9207 seconds
59
60 TACC: Shutdown complete. Exiting.
61 TACC: Starting up job 5012959
62 TACC: Setting up parallel environment for MMAPICH2+mpispaaw.
63 TACC: Starting parallel tasks...
64
65 Strong scaling estimates are :
66 1.27 1.50 1.64 1.61 2.27 (speedup)
67 1.27 0.75 0.41 0.20 0.14 (efficiency) for
68 1 2 4 8 16 threads/processors
69
70 Average strong scaling efficiency: 0.55
71
72 Weak scaling estimates are :
73 1.27 0.74 0.42 0.24 0.12 (efficiency) for
74 1 2 4 8 16 threads/processors
75
76 Average weak scaling efficiency: 0.56
77
78
79 mpi Grade = 79.76
```

```
2 n = 5000,threads = 1, simulation time = 0.465573 seconds
3 n = 5000,threads = 2, simulation time = 0.541029 seconds
4 n = 5000,threads = 4, simulation time = 0.283693 seconds
5 n = 5000,threads = 8, simulation time = 0.146827 seconds
6 n = 5000,threads = 16, simulation time = 0.389091 seconds
7 n = 10000,threads = 2, simulation time = 1.13281 seconds
8 n = 20000,threads = 4, simulation time = 1.19961 seconds
9 n = 40000,threads = 8, simulation time = 1.1629 seconds
10 n = 80000,threads = 16, simulation time = 2.8414 seconds
11
12 Strong scaling estimates are :
13 1.59 1.37 2.62 5.06 1.91 (speedup)
14 1.59 0.69 0.65 0.63 0.12 (efficiency) for
15 1 2 4 8 16 threads/processors
16
17 Average strong scaling efficiency: 0.74
18
19 Weak scaling estimates are :
20 1.59 0.66 0.62 0.64 0.26 (efficiency) for
21 1 2 4 8 16 threads/processors
22
23 Average weak scaling efficiency: 0.75
24
25
26 openmp Grade = 95.44
```

I will now briefly outline the difficulties and methods for my Barnes Hut and binning approaches below.

BARNES HUT

I had first created a Barnes-Hut Tree class that constructed a Barnes-Hut quadtree, which is a quadtree that groups particles that are "sufficiently nearby" and recursively divides the set of particles into groups by using a quadtree. It can apply the net forces to N particles in the tree, and does so more efficiently than brute-force methods by using the center of mass of "far quadrants" so we don't have to look at all N particles when calculating the net force.

Although I enjoyed working with this recursive data structure, the Barnes-Hut quadtree failed to scale well to large values of N (I had to rebuild the tree every time) and the MPI part of the assignment did not lend itself to a simple solution since we could not share the quadtree easily. Thus, I “bit-the-bullet”, so to speak, and started over the homework using a vector of bins of particles instead of a quadtree.

BINNING

The binning approach involved dividing the world into bins and uses only the particles in surrounding bins to apply forces to each particle in a given bin. Originally wanting to create an object that managed my bins data-structure, I kept all of the methods and the vector of a lists of particles self-contained in the Serial, OpenMP, and MPI cpp files because the overhead for a class was far too expensive.

As this affects the grades of OpenMP and MPI implementations, I had to try to make the Serial code inefficient while still avoiding performing n^2 calculations on the particles. The Serial implementation did the following:

- The bins are vectors of lists
- Bins are reset every iteration
- A check is being done if the current particle is itself for every force computation
- It is iterating over bins instead of particles
- The bins aren't being looped over in contiguous memory

The OpenMP grade, on the other hand, relies on efficiency, and so that implementation did the following:

- The bins are vectors of lists
- Bins are only rebuilt, and not reset, every iteration
- They iterate over particles and not the bins themselves (this avoids using expensive iterators)
- When it does have to iterate over the bins (i.e. when it is rebuilding all of the bins), much care is taken to make sure that we are iterating over contiguous memory
- Force calculation, movement of particles, and the rebuilding of bins are all done with parallel for loops

I had the most difficulty with the MPI portion, and, after a great deal of struggling, I found the best performance by simply doing what we did with OpenMP except now each thread has its own copy of their bins. I believe a better performance could be achieved if we could divide the bins instead of the particles to each thread, but coming up with a complex datatype like a bin along with avoiding particles disappearing at thread boundaries proved a little too difficult (and it resulted in many seg-faults that Valgrind couldn't help me diagnose!)

CONCLUSION

Although I struggled a great deal with this assignment (and basically having to do it twice!), I enjoyed learning about and using quadrees, C++ vectors, OpenMP, and MPI. Admittedly, I prefer using OpenMP over MPI and PThreads due to its ease of use, I feel that having some experience with MPI is a good thing to have for any programmer and thus I am happy to have the experience.